

Stock Price Prediction

July 21, 2025

1 Stock_Price_Prediction_Model

```
[4]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import yfinance as yf
```

```
[5]: start = '2015-01-01'
end = '2025-07-10'
stock = 'MSFT'

data = yf.download(stock, start, end)
```

C:\Users\DELL\AppData\Local\Temp\ipykernel_4908\914705923.py:5: FutureWarning:
YF.download() has changed argument auto_adjust default to True
data = yf.download(stock, start, end)
[*****100%*****] 1 of 1 completed

```
[6]: data
```

```
[6]: Price          Close          High          Low          Open          Volume
Ticker           MSFT           MSFT           MSFT           MSFT           MSFT
Date
2015-01-02    39.998699    40.563266    39.810512    39.913160    27913900
2015-01-05    39.630894    39.973054    39.562460    39.665107    39673900
2015-01-06    39.049206    39.990149    38.955111    39.673650    36447900
2015-01-07    39.545334    39.742076    38.912337    39.331483    29114100
2015-01-08    40.708687    40.845551    39.964486    39.990147    29645200
...
2025-07-02    491.089996    493.500000    488.700012    489.989990    16319600
2025-07-03    498.839996    500.130005    493.440002    493.809998    13984800
2025-07-07    497.720001    498.750000    495.230011    497.380005    13981600
2025-07-08    496.619995    498.200012    494.109985    497.239990    11846600
2025-07-09    503.510010    506.779999    499.739990    500.299988    18659500
```

[2644 rows x 5 columns]

```
[7]: data.reset_index(inplace=True)
```

```
[8]: data
```

```
[8]: Price      Date      Close      High      Low      Open      Volume
Ticker      MSFT      MSFT      MSFT      MSFT      MSFT
0      2015-01-02      39.998699      40.563266      39.810512      39.913160      27913900
1      2015-01-05      39.630894      39.973054      39.562460      39.665107      39673900
2      2015-01-06      39.049206      39.990149      38.955111      39.673650      36447900
3      2015-01-07      39.545334      39.742076      38.912337      39.331483      29114100
4      2015-01-08      40.708687      40.845551      39.964486      39.990147      29645200
...      ...      ...      ...      ...      ...
2639      2025-07-02      491.089996      493.500000      488.700012      489.989990      16319600
2640      2025-07-03      498.839996      500.130005      493.440002      493.809998      13984800
2641      2025-07-07      497.720001      498.750000      495.230011      497.380005      13981600
2642      2025-07-08      496.619995      498.200012      494.109985      497.239990      11846600
2643      2025-07-09      503.510010      506.779999      499.739990      500.299988      18659500
```

```
[2644 rows x 6 columns]
```

```
[9]: data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2644 entries, 0 to 2643
Data columns (total 6 columns):
#   Column      Non-Null Count  Dtype
---  -
0   (Date, )    2644 non-null  datetime64[ns]
1   (Close, MSFT) 2644 non-null  float64
2   (High, MSFT) 2644 non-null  float64
3   (Low, MSFT) 2644 non-null  float64
4   (Open, MSFT) 2644 non-null  float64
5   (Volume, MSFT) 2644 non-null  int64
dtypes: datetime64[ns](1), float64(4), int64(1)
memory usage: 124.1 KB
```

```
[10]: data.columns = ['_'.join(col).strip() for col in data.columns]
```

```
[11]: print(data.columns)
```

```
Index(['Date_', 'Close_MSFT', 'High_MSFT', 'Low_MSFT', 'Open_MSFT',
      'Volume_MSFT'],
      dtype='object')
```

```
[12]: data = data.rename(columns={'Date_': 'Date'})
```

```
[13]: data
```

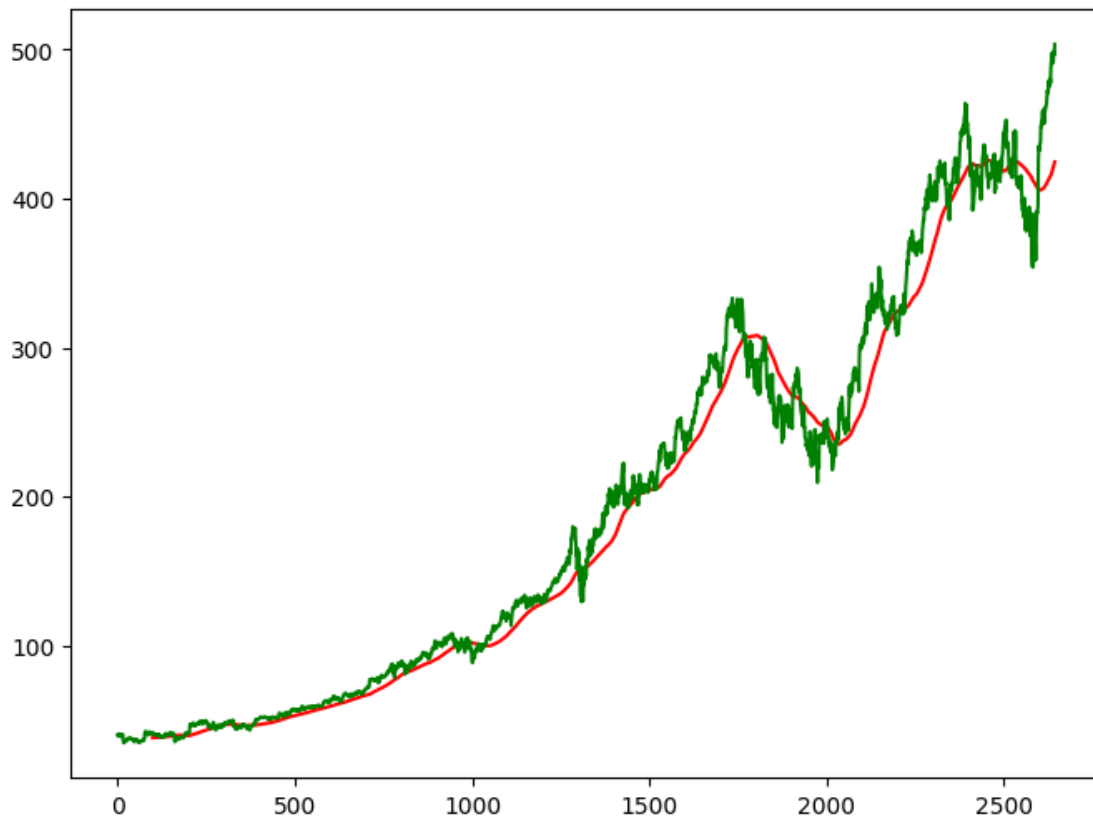
```
[13]:      Date      Close_MSFT      High_MSFT      Low_MSFT      Open_MSFT      Volume_MSFT
0      2015-01-02      39.998699      40.563266      39.810512      39.913160      27913900
```

1	2015-01-05	39.630894	39.973054	39.562460	39.665107	39673900
2	2015-01-06	39.049206	39.990149	38.955111	39.673650	36447900
3	2015-01-07	39.545334	39.742076	38.912337	39.331483	29114100
4	2015-01-08	40.708687	40.845551	39.964486	39.990147	29645200
...	...	...	...	...	...	...
2639	2025-07-02	491.089996	493.500000	488.700012	489.989990	16319600
2640	2025-07-03	498.839996	500.130005	493.440002	493.809998	13984800
2641	2025-07-07	497.720001	498.750000	495.230011	497.380005	13981600
2642	2025-07-08	496.619995	498.200012	494.109985	497.239990	11846600
2643	2025-07-09	503.510010	506.779999	499.739990	500.299988	18659500

[2644 rows x 6 columns]

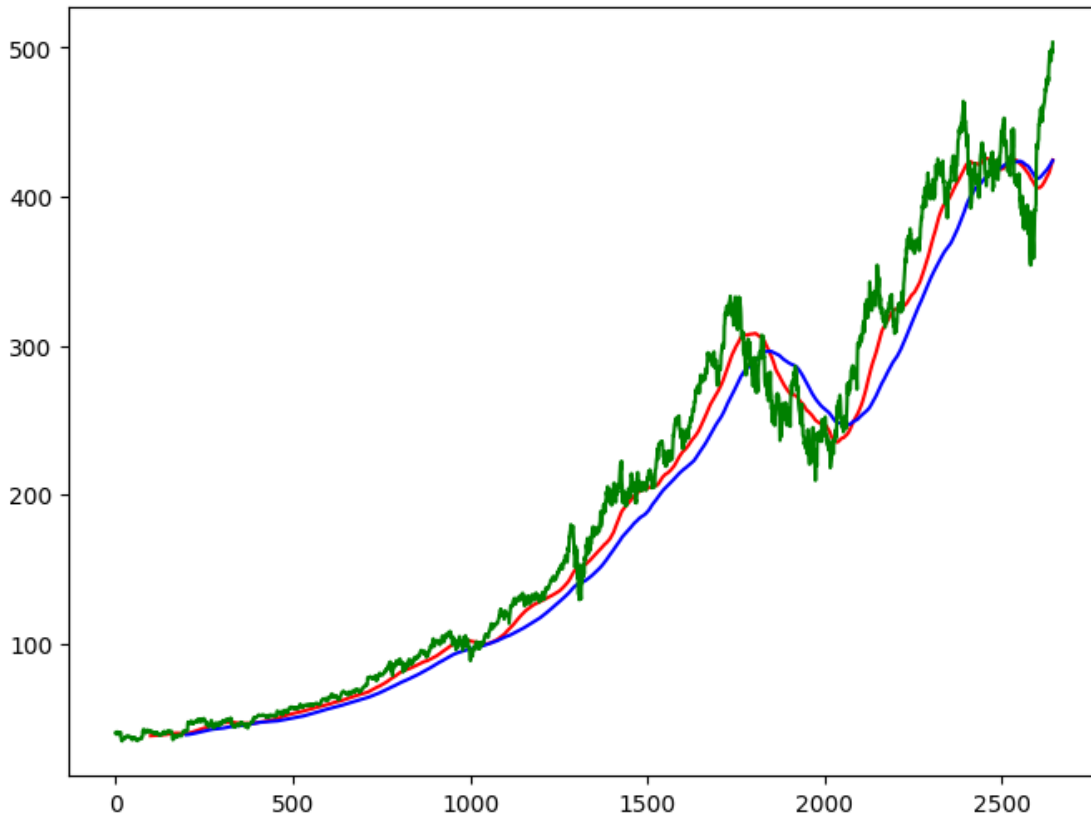
```
[14]: ma_100_days = data.Close_MSFT.rolling(100).mean()
```

```
[15]: plt.figure(figsize=(8,6))
plt.plot(ma_100_days, 'r')
plt.plot(data.Close_MSFT, 'g')
plt.show()
```



```
[16]: ma_200_days = data.Close_MSFT.rolling(200).mean()
```

```
[17]: plt.figure(figsize=(8,6))
plt.plot(ma_100_days, 'r')
plt.plot(ma_200_days, 'b')
plt.plot(data.Close_MSFT, 'g')
plt.show()
```



```
[18]: data.dropna(inplace=True)
```

```
[19]: pd.isnull(data).sum()
```

```
[19]: Date          0
Close_MSFT        0
High_MSFT         0
Low_MSFT          0
Open_MSFT         0
Volume_MSFT       0
dtype: int64
```

```
[20]: data_train = pd.DataFrame(data.Close_MSFT[0: int(len(data)*0.80)])
data_test  = pd.DataFrame(data.Close_MSFT[int(len(data)*0.80): len(data)])
```

```
[21]: data_train.shape[0]
```

```
[21]: 2115
```

```
[22]: data_test.shape[0]
```

```
[22]: 529
```

```
[23]: from sklearn.preprocessing import MinMaxScaler  
scaler = MinMaxScaler(feature_range=(0,1))
```

```
[24]: data_train_scale = scaler.fit_transform(data_train)
```

```
[25]: x = []  
y = []  
  
for i in range(100, data_train_scale.shape[0]):  
    x.append(data_train_scale[i-100:i])  
    y.append(data_train_scale[i,0])
```

```
[26]: x, y = np.array(x), np.array(y)
```

```
[27]: from keras.layers import Dense, Dropout, LSTM  
from keras.models import Sequential
```

```
[28]: model = Sequential()  
model.add(LSTM(units = 50, activation = 'relu', return_sequences = True,  
    ↪ input_shape = (x.shape[1],1)))  
model.add(Dropout(0.2))  
  
model.add(LSTM(units = 60, activation = 'relu', return_sequences = True))  
model.add(Dropout(0.3))  
  
model.add(LSTM(units = 80, activation = 'relu', return_sequences = True))  
model.add(Dropout(0.4))  
  
model.add(LSTM(units = 120, activation = 'relu'))  
model.add(Dropout(0.5))  
  
model.add(Dense(units = 1))
```

```
C:\Users\DELL\anaconda3\envs\tf_env\lib\site-  
packages\keras\src\layers\rnn\rnn.py:199: UserWarning: Do not pass an  
`input_shape`/`input_dim` argument to a layer. When using Sequential models,  
prefer using an `Input(shape)` object as the first layer in the model instead.  
super().__init__(**kwargs)
```

```
[29]: model.compile(optimizer = 'adam', loss = 'mean_squared_error')
```

```
[30]: model.fit(x,y, epochs = 50, batch_size = 32, verbose = 1)
```

```
Epoch 1/50
63/63          52s 500ms/step -
loss: 0.0952
Epoch 2/50
63/63          31s 490ms/step -
loss: 0.0110
Epoch 3/50
63/63          40s 474ms/step -
loss: 0.0076
Epoch 4/50
63/63          41s 471ms/step -
loss: 0.0071
Epoch 5/50
63/63          42s 493ms/step -
loss: 0.0060
Epoch 6/50
63/63          30s 477ms/step -
loss: 0.0054
Epoch 7/50
63/63          42s 489ms/step -
loss: 0.0061
Epoch 8/50
63/63          32s 508ms/step -
loss: 0.0049
Epoch 9/50
63/63          29s 464ms/step -
loss: 0.0057
Epoch 10/50
63/63          43s 501ms/step -
loss: 0.0049
Epoch 11/50
63/63          29s 462ms/step -
loss: 0.0049
Epoch 12/50
63/63          42s 477ms/step -
loss: 0.0042
Epoch 13/50
63/63          30s 482ms/step -
loss: 0.0043
Epoch 14/50
63/63          30s 481ms/step -
loss: 0.0045
Epoch 15/50
63/63          30s 476ms/step -
loss: 0.0042
Epoch 16/50
```

63/63	30s 479ms/step -
loss: 0.0044	
Epoch 17/50	
63/63	41s 476ms/step -
loss: 0.0044	
Epoch 18/50	
63/63	29s 456ms/step -
loss: 0.0038	
Epoch 19/50	
63/63	42s 466ms/step -
loss: 0.0037	
Epoch 20/50	
63/63	30s 475ms/step -
loss: 0.0040	
Epoch 21/50	
63/63	41s 476ms/step -
loss: 0.0047	
Epoch 22/50	
63/63	30s 470ms/step -
loss: 0.0045	
Epoch 23/50	
63/63	40s 458ms/step -
loss: 0.0033	
Epoch 24/50	
63/63	41s 461ms/step -
loss: 0.0035	
Epoch 25/50	
63/63	29s 461ms/step -
loss: 0.0035	
Epoch 26/50	
63/63	29s 459ms/step -
loss: 0.0035	
Epoch 27/50	
63/63	30s 479ms/step -
loss: 0.0032	
Epoch 28/50	
63/63	40s 457ms/step -
loss: 0.0030	
Epoch 29/50	
63/63	29s 458ms/step -
loss: 0.0032	
Epoch 30/50	
63/63	30s 469ms/step -
loss: 0.0033	
Epoch 31/50	
63/63	31s 485ms/step -
loss: 0.0028	
Epoch 32/50	

63/63	40s 472ms/step -
loss: 0.0031	
Epoch 33/50	
63/63	31s 489ms/step -
loss: 0.0030	
Epoch 34/50	
63/63	32s 516ms/step -
loss: 0.0032	
Epoch 35/50	
63/63	32s 501ms/step -
loss: 0.0034	
Epoch 36/50	
63/63	33s 530ms/step -
loss: 0.0029	
Epoch 37/50	
63/63	30s 477ms/step -
loss: 0.0028	
Epoch 38/50	
63/63	40s 465ms/step -
loss: 0.0029	
Epoch 39/50	
63/63	28s 451ms/step -
loss: 0.0027	
Epoch 40/50	
63/63	42s 466ms/step -
loss: 0.0027	
Epoch 41/50	
63/63	41s 463ms/step -
loss: 0.0027	
Epoch 42/50	
63/63	43s 485ms/step -
loss: 0.0028	
Epoch 43/50	
63/63	39s 462ms/step -
loss: 0.0030	
Epoch 44/50	
63/63	29s 460ms/step -
loss: 0.0029	
Epoch 45/50	
63/63	31s 492ms/step -
loss: 0.0024	
Epoch 46/50	
63/63	31s 485ms/step -
loss: 0.0027	
Epoch 47/50	
63/63	40s 470ms/step -
loss: 0.0028	
Epoch 48/50	


```

63/63          41s 473ms/step -
loss: 0.0029
Epoch 49/50
63/63          40s 461ms/step -
loss: 0.0027
Epoch 50/50
63/63          30s 472ms/step -
loss: 0.0026

```

[30]: <keras.src.callbacks.history.History at 0x264485fc7c0>

[31]: `model.summary()`

Model: "sequential"

Layer (type) ↳ Param #	Output Shape	
lstm (LSTM) ↳ 10,400	(None, 100, 50)	↳
dropout (Dropout) ↳ 0	(None, 100, 50)	↳
lstm_1 (LSTM) ↳ 26,640	(None, 100, 60)	↳
dropout_1 (Dropout) ↳ 0	(None, 100, 60)	↳
lstm_2 (LSTM) ↳ 45,120	(None, 100, 80)	↳
dropout_2 (Dropout) ↳ 0	(None, 100, 80)	↳
lstm_3 (LSTM) ↳ 96,480	(None, 120)	↳
dropout_3 (Dropout) ↳ 0	(None, 120)	↳
dense (Dense) ↳ 121	(None, 1)	↳

Total params: 536,285 (2.05 MB)

Trainable params: 178,761 (698.29 KB)

Non-trainable params: 0 (0.00 B)

Optimizer params: 357,524 (1.36 MB)

```
[65]: pas_100_days = data_train.tail(100)
```

```
[69]: data_test = pd.concat([pas_100_days, data_test], ignore_index = True)
```

```
[71]: data_test_scale = scaler.fit_transform(data_test)
```

```
[73]: x = []
      y = []

      for i in range(100, data_test_scale.shape[0]):
          x.append(data_test_scale[i-100:i])
          y.append(data_test_scale[i,0])
```

```
[75]: x, y = np.array(x), np.array(y)
```

```
[76]: y_predict = model.predict(x)
```

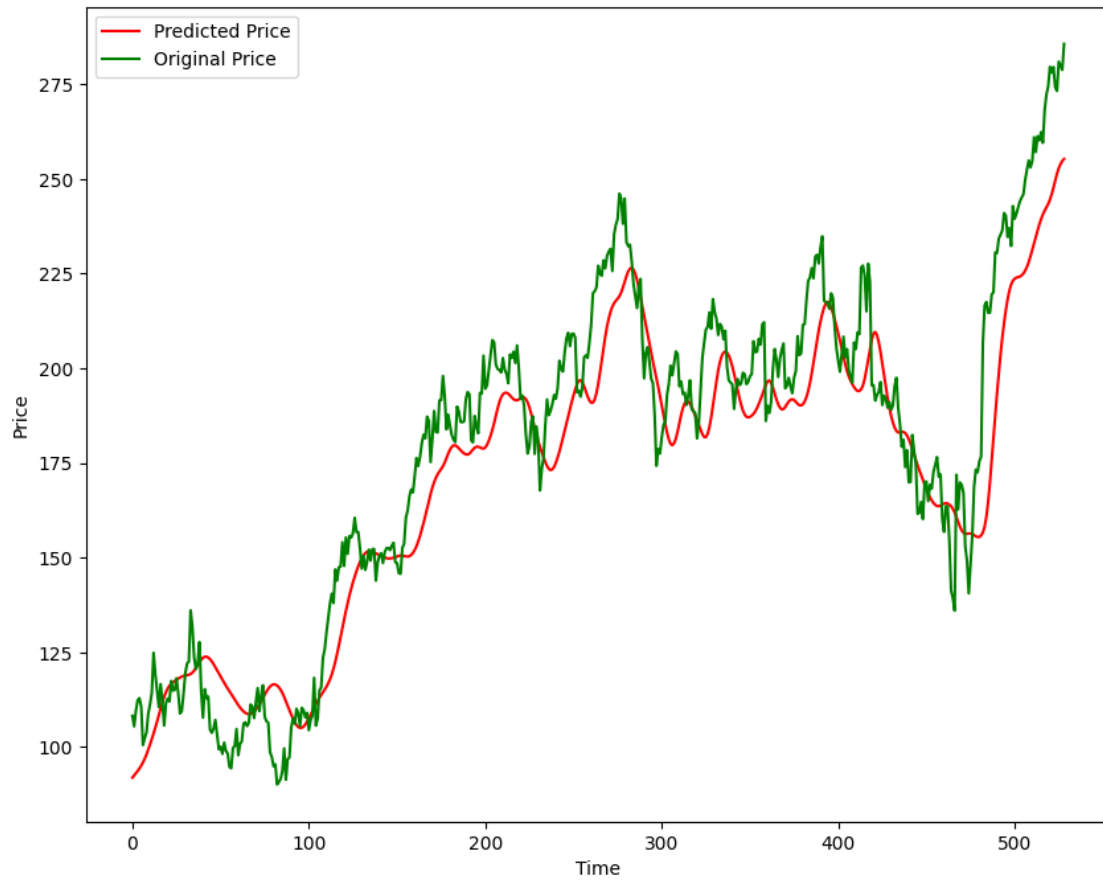
17/17 7s 311ms/step

```
[77]: scale = 1/scaler.scale_
```

```
[78]: y_predict = y_predict*scale
```

```
[79]: y = y*scale
```

```
[80]: plt.figure(figsize=(10,8))
      plt.plot(y_predict, 'r', label='Predicted Price')
      plt.plot(y, 'g', label='Original Price')
      plt.xlabel('Time')
      plt.ylabel('Price')
      plt.legend()
      plt.show()
```



```
[85]: model.save('Stock Predictions Model.keras')
```